Graceful Degradation and Fallbacks

less, but not nothing

Watch the 4-min video

In one line: decide in advance what every non-critical feature does when its dependency fails (stale data, a default, a static page, or simply hiding it) so a partial failure removes a feature instead of the whole page, and shed expensive features on purpose to protect the critical path.

1 Same failure, two outcomes

The personalisation service fails. Option one: the home page throws an error and nobody can shop. Option two: the home page shows popular items and most users never notice. The failure is identical; the design is different. **Graceful degradation** means serving reduced functionality instead of failing completely.

2 Critical vs non-critical

Classify features first. **Critical**: login, cart, checkout, payment: must work, and get the strongest protection. **Non-critical**: recommendations, reviews, recently viewed, live stock counts: can be simplified or dropped. Then, for each non-critical feature, write down its fallback, its timeout (lesson 4) and its circuit breaker (lesson 6).

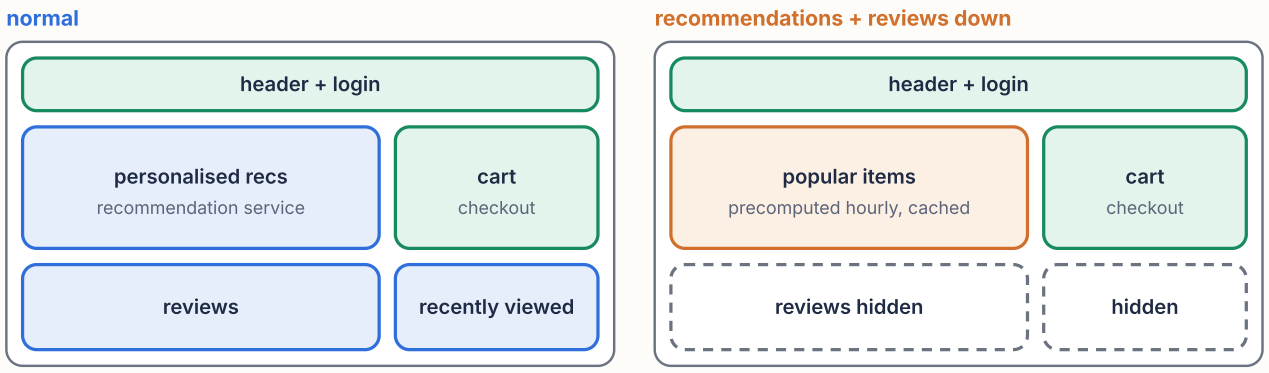


Figure 1. The page is composed of independent parts. When two dependencies fail, those parts degrade and the critical path is untouched.

3 Kinds of fallback

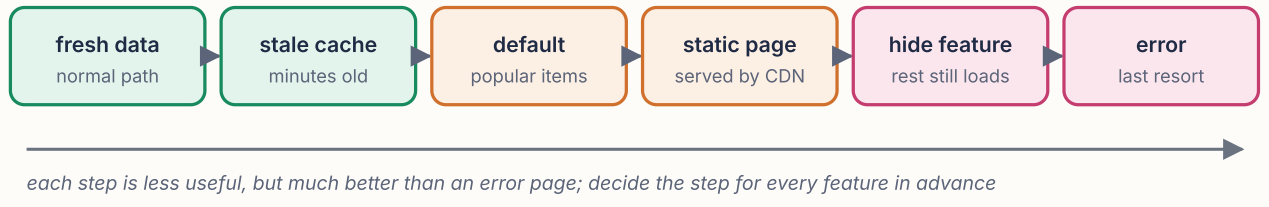


Figure 2. Fallbacks from best to worst. Most features should never reach the last box.

Fallback	Example	Watch out for
Stale / cached data	prices from a 5-minute-old cache	must be safe to be stale; never stale balances or stock at checkout
Default value	popular items, generic banner, default settings	precompute it; the default must not need the failed service
Static version	CDN-served static home page	keep it reasonably current
Hide the feature	no reviews section	page layout must cope with a missing block
Feature shedding	turn off search suggestions at peak	needs a fast switch: a feature flag (lesson 76), not a deploy

Feature shedding is degrading on purpose under load: during a sales peak, switch off search suggestions, personalised ranking and live inventory counts so checkout gets all the capacity.

4 "The cache cluster died. Now what?"

Without a plan every request falls through to the database, which was sized for the cache's 5% miss rate, not 100%, and collapses. A better design: **limit concurrent database reads** with a bulkhead (lesson 7) and reject the excess; serve **stale data from a small in-process cache**; **shed** non-critical features; return **defaults** where data isn't essential; and when the cache comes back, **warm it gradually** (request coalescing, ramped traffic) to avoid a stampede.

WORKED EXAMPLE: POPULAR ITEMS WHEN RECOMMENDATIONS FAIL

The home page calls the recommendation service with a **200 ms timeout** behind a circuit breaker. On timeout, error or open breaker it shows **popular items**, precomputed every hour by a batch job and cached locally, so the fallback needs nothing from the failed service. The page loads normally and sales dip slightly instead of dropping to zero. A `degraded_mode` metric shows how often the fallback serves; when the service recovers, personalised results quietly return.

COMMON MISTAKES

- Fallbacks never tested, so they fail the first time they run. Exercise them with chaos experiments and game days (lesson 17).
- A complex fallback that depends on another service, which can fail too.
- No metric for degraded mode, so a fallback silently serves for weeks.
- Degrading silently where users need to know: say "recommendations are temporarily unavailable".

5 Key terms

Graceful degradation: reduced functionality instead of total failure

Critical path: features that must work: login, cart, checkout

Stale data: older cached data served when fresh data is unavailable

Fallback: pre-planned response when a dependency fails

Feature shedding: turning off non-critical features under load

Cache stampede: many misses hitting the database at once

6 Interview questions

Q1. What is graceful degradation? BASIC
Serving reduced functionality (cached data, defaults, static pages, hidden features) instead of failing completely.

Q2. Name four kinds of fallback. BASIC
Stale cached data, a sensible default (e.g. popular items), a static CDN-served page, and hiding the feature.

Q3. How would your system degrade if the cache cluster died? SCENARIO
Protect the database with concurrency limits, serve stale data from local in-process caches, shed non-critical features, return defaults where data isn't essential, and warm the cache back gradually to avoid a stampede.

Q4. Black Friday traffic is 5× normal and checkout slows. What do you switch off? SCENARIO
Expensive non-critical features via feature flags: search suggestions, personalised ranking, live stock counts, so their capacity goes to checkout.

Q5. Why must a fallback be simple? TRICKY
It runs exactly when things are already failing; if it needs another service or complex logic, it can fail too and turn a partial outage into a full one.

Q6. Can you serve stale data everywhere? TRICKY
No: stale recommendations are fine, but stale balances, stock at payment or permissions can cause real harm; decide per feature.

Next: SD-K-009 · Health Checks and Self-Healing backendroadmap.com